

SENDA: ruteo de vehículos con búsqueda de trayectoria única, explicado y medido con honestidad

Guía educativa del núcleo de ruteo de LATTIMEX: cómo funciona, cómo se evaluó
y cómo probarlo en su computadora

Erick Martínez

LATTIMEX · investigación independiente · Querétaro, México
contacto@lattimex.com · <https://lattimex.com>

30 de septiembre de 2026

Resumen

En pocas palabras. Repartir entregas entre vehículos y decidir el orden de visita es un problema clásico de optimización que no se puede resolver exactamente a gran escala: se usan heurísticas, programas que buscan muy buenas soluciones en poco tiempo. Esta guía abre una de ellas, el núcleo de ruteo del motor LATTIMEX, pieza por pieza, y muestra cómo medir su calidad sin engañarse. Al final, el lector puede instalar el motor, que es de código abierto (MIT), y comprobar los resultados en su propia computadora.

Este documento explica, desde cero y con ejemplos resueltos, cómo funciona y cómo se evaluó SENDA, un solver para el problema de ruteo de vehículos con capacidades (CVRP). El solver sigue una sola trayectoria de búsqueda: construye una solución inicial, la mejora con búsqueda local granular, la reestructura con una búsqueda adaptativa de vecindarios grandes (ALNS), la refina con movimientos de segmento y SWAP*, y reparte su tiempo entre dos o tres variantes elegidas según los rasgos de la instancia (un portafolio de configuraciones). Para cada componente se explica qué es, por qué importa y cuánto aporta. Un experimento de cascada mide la calidad disponible después de cada etapa: en A/B/E el gap medio pasa de 81.008 % tras la construcción a 0.115 % con el portafolio completo, y en X de 25.786 % a 4.121 %. Con un contrato congelado, 10 réplicas externas, el mismo reloj para todos los métodos y verificación independiente, SENDA obtiene en 60 instancias A/B/E 0.111 % de gap frente a 0.004 % de HGS-CVRP (empata en 37 y HGS es mejor en 23). En 41 instancias X obtiene 4.119 %, frente a 1.450 % de HGS y 0.758 % de FILO2. El documento incluye un caso de estudio sobre un error de verificación y su corrección.

Palabras clave: ruteo de vehículos; CVRP; ALNS; búsqueda granular; SWAP*; evaluación reproducible; docencia en optimización.

Índice

Cómo leer este documento	3
1. Introducción	4
2. El CVRP en diez minutos	4
2.1. Definición	4
2.2. Cómo se mide la calidad: BKS y gap	5
2.3. Los bancos de prueba	5
3. Qué es SENDA: una trayectoria de búsqueda por corrida	5

4. Mapa del solver: cómo fluye una solución	6
5. Los componentes en detalle	7
5.1. Construcción inicial	7
5.2. Búsqueda local granular	7
5.3. ALNS con recocido simulado	8
5.4. Movimientos de segmento y Or-opt-3	10
5.5. SWAP*	10
5.6. Intensificación del mejor global	11
5.7. Selector FAST	11
5.8. El portafolio: la configuración evaluada	11
5.9. Parámetros	12
6. Cómo se acumula la calidad: el experimento de cascada	12
7. Complejidad computacional: cuánto cuesta cada paso	14
7.1. La notación O grande	14
7.2. Por qué el CVRP no se resuelve exactamente	15
7.3. Notación para el análisis	16
7.4. Costo de cada componente	16
7.5. Comprobación experimental	17
7.6. Dónde podría bajar el costo	18
8. Cómo evaluar una heurística sin engañarse	18
9. Resultados confirmatorios	19
9.1. A/B/E: casi óptimo, pero HGS es sistemáticamente mejor	19
9.2. X: la arquitectura ligera deja de ser competitiva	20
9.3. Ablación: qué sostiene la calidad	20
9.4. Holdout prospectivo	21
10. Caso de estudio: una auditoría que corrigió nuestros propios resultados	21
10.1. Qué se publicó	21
10.2. Cómo se detectó el problema	22
10.3. La causa	22
10.4. La corrección	22
11. Discusión	23
12. Conclusiones	23
13. Pruébalo en su computadora	24
A. Glosario	26
B. Regla literal del selector FAST	27
C. Errata respecto a la versión 2	27

Cómo leer este documento

El texto está dirigido a estudiantes, ingenieros de logística y cualquier persona que quiera entender una heurística moderna de ruteo y la forma correcta de medir su calidad. No se requiere experiencia previa con metaheurísticas: basta con nociones de grafos, probabilidad y estadística básica.

- Las Secciones 2 y 3 presentan el problema y qué es SENDA.
- Las Secciones 4–6 son el núcleo didáctico: describen cada componente, cómo pasa la solución de uno a otro y cuánta calidad agrega cada paso.
- La Sección 7 introduce la notación O grande y estima cuánto cuesta cada componente.
- La Sección 8 explica cómo evaluar una heurística sin engañarse.
- Las Secciones 9 y 10 presentan los resultados y un caso de estudio sobre un error real.
- La Sección 13 explica cómo instalar el motor y repetir las mediciones en su computadora.

Qué aprenderá. (1) Qué es el CVRP y por qué se resuelve con heurísticas; (2) cómo se construye, mejora y diversifica una solución de ruteo, con un ejemplo para cada componente; (3) cómo estimar con la notación O grande cuánto cuesta cada componente y por qué eso limita la calidad con un reloj fijo; (4) cómo diseñar una comparación justa (mismo reloj, réplicas, verificación independiente, estadística pareada); (5) cómo un error de verificación puede fabricar “victorias” y cómo detectarlo; y (6) cómo pasar de un solver académico a un motor que planifica rutas reales con carga tridimensional.

Los recuadros tienen cuatro funciones:

Idea clave

Resume el concepto que no debe olvidarse.

Ejemplo resuelto

Resuelve un cálculo pequeño, paso a paso y con números concretos.

Lección metodológica

Describe un error posible, o uno que realmente ocurrió en este proyecto, y cómo evitarlo.

Para reproducir

Indica las huellas SHA-256 de los artefactos que respaldan una cifra y cómo repetir la medición.

Sobre esta edición. Esta guía acompaña al repositorio público de LATTIMEX. Los programas usados para comparar (HGS-CVRP, FILO2 y los scripts de las campañas) no forman parte del repositorio: se describen aquí para que la comparación sea transparente, y la Sección 13 explica cómo repetirla con copias obtenidas de sus fuentes oficiales. Los datos crudos, contratos y bitácoras se conservan en el archivo del autor; cada cifra cita la huella SHA-256 del artefacto que la respalda. Esta edición incluye la corrección de la comparación A/B/E (Sección 10).

1. Introducción

Todos los días, empresas de paquetería, distribución y servicios deciden qué vehículo visita a qué clientes y en qué orden. Esa decisión, repetida miles de veces, es el *problema de ruteo de vehículos con capacidades* (CVRP), formulado por primera vez por Dantzig y Ramser [5]. Su enunciado es breve, pero el número de soluciones posibles crece de forma explosiva con el número de clientes. Por eso, en la práctica se resuelve con *heurísticas*: algoritmos que encuentran soluciones muy buenas en segundos, aunque no pueden garantizar que sean óptimas.

El estado del arte en heurísticas está dominado por la búsqueda genética híbrida HGS-CVRP [16, 15] y por solvers muy rápidos como FILO2 [1]. Este trabajo no pretende superarlos. Su objetivo es construir un solver *compacto y auditable* —una sola trayectoria de búsqueda, sin población de soluciones ni cruce— y responder con evidencia reproducible tres preguntas:

P1. ¿Qué hace cada componente del solver y cuánta calidad agrega?

P2. ¿Qué calidad final alcanza, con el mismo tiempo que los solvers de referencia, en instancias pequeñas (A/B/E) y grandes (X)?

P3. ¿Qué se aprende cuando el propio protocolo de evaluación contiene un error?

Contribuciones. (1) Una descripción completa de SENDA, componente por componente, con pseudocódigo y ejemplos numéricos. (2) Una medición de la calidad que agrega cada etapa, complementaria a la ablación clásica. (3) Un protocolo de evaluación con contrato congelado, réplicas, verificación independiente e inferencia por instancia. (4) Una comparación honesta frente a HGS y FILO2 que delimita dónde la arquitectura ligera es competitiva y dónde no. (5) Un caso de estudio sobre un error de verificación y su corrección.

2. El CVRP en diez minutos

2.1. Definición

Una instancia del CVRP tiene un depósito, identificado con el nodo 0; un conjunto de clientes $V = \{1, \dots, n\}$, cada uno con una demanda $q_i > 0$; una capacidad vehicular Q ; y una distancia $d(i, j)$ entre cada par de puntos. Una *ruta* sale del depósito, visita una secuencia de clientes y regresa al depósito. Una *solución* es un conjunto de rutas que cumple tres condiciones:

- (I) cada cliente se visita exactamente una vez (*cobertura*);
- (II) la demanda total de cada ruta no excede Q (*capacidad*);
- (III) el número de rutas no excede la flota disponible k , cuando la instancia la fija (*flota*).

El objetivo es minimizar la distancia total recorrida:

$$C(S) = \sum_{r \in S} \sum_{(i,j) \in r} d(i,j). \quad (1)$$

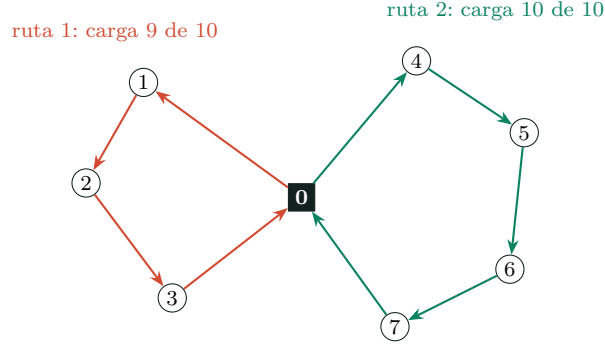


Figura 1: Solución de un CVRP pequeño con 7 clientes, capacidad $Q = 10$ y flota $k = 2$. Cada cliente se visita una sola vez y ninguna ruta excede la capacidad.

2.2. Cómo se mide la calidad: BKS y gap

Para las instancias de referencia de CVRPLIB [4] se conoce la *mejor solución conocida* (BKS, por sus siglas en inglés); muchas de ellas están certificadas como óptimas [10]. La calidad de una heurística se mide con el *gap*, la diferencia relativa entre su costo y la BKS:

$$\text{gap}(S) = 100 \cdot \frac{C(S) - C_{\text{BKS}}}{C_{\text{BKS}}} (\%). \quad (2)$$

Ejemplo resuelto: calcular un gap

En la instancia B-n51-k7 la BKS vale 1032 y usa $k = 7$ vehículos. Si un solver entrega una solución de costo 1040 con 7 rutas, su gap es $100 \cdot (1040 - 1032)/1032 = 0.78 \%$. Si entrega una de costo 1032, su gap es 0 %: alcanzó la BKS.

Idea clave: un gap negativo es una alarma

Si la BKS está certificada como óptima, *ningún* solver correcto puede obtener un gap negativo. Un gap negativo indica que la solución viola alguna restricción —casi siempre la de flota— o que la verificación está mal hecha. La Sección 10 muestra que esto ocurrió en este proyecto.

2.3. Los bancos de prueba

- **A, B y E** (Augerat y Christofides [2, 3]): 63 instancias clásicas de 12 a 100 clientes, con flota fija. El nombre indica el tamaño y la flota: A-n32-k5 tiene 32 nodos (31 clientes y el depósito) y 5 vehículos.
- **X** (Uchoa et al. [14]): instancias grandes y más variadas. Aquí se usan 41, de 199 a 428 clientes, con flota libre. En X, el número k del nombre es una referencia y no una restricción.

3. Qué es SENDA: una trayectoria de búsqueda por corrida

SENDA (*Single-trajectory ENgine with Destroy-and-repair Adaptive search*; en español, “sendero”) es el núcleo de ruteo del motor LATTIMEX. Su rasgo de diseño es el que le da nombre: sigue **una sola trayectoria de búsqueda por corrida**. Dentro de cada corrida hay en todo momento una solución actual —con todas sus rutas— que se destruye parcialmente, se repara y se mejora; no hay una población de soluciones que se crucen entre sí, como en los algoritmos genéticos del estado del arte para el CVRP (por ejemplo, HGS-CVRP de Vidal [15]).

La decisión tiene ventajas prácticas que importan en un motor de producción: el recorrido de la solución se puede seguir paso a paso y explicar (Sección 4); el esfuerzo se reparte con un

reloj explícito entre pocas etapas; y el paralelismo se obtiene con un portafolio de trayectorias independientes. La elección final entre sus candidatos es determinista; como el presupuesto de cada corrida es tiempo de reloj y depende del número de hilos, los candidatos —y por tanto la respuesta— pueden variar entre computadoras. El costo también es claro, y esta guía lo mide: en el CVRP puro, con el mismo reloj, un algoritmo poblacional bien afinado sigue siendo más preciso (Sección 9).

SENDa combina ideas publicadas —ALNS [11], vecindarios granulares [13], infactibilidad penalizada y el vecindario SWAP* [15]— en una implementación propia. La aportación de esta implementación está en cómo se articulan: el selector de configuración por rasgos de la instancia, el portafolio de brazos con el reloj repartido y, en el motor LATTIMEX, la integración con el acomodo tridimensional de la carga, la mediación con flota fija y el plan de descarga por puertas.

4. Mapa del solver: cómo fluye una solución

Antes de estudiar cada componente conviene ver el recorrido completo. En SENDa circula un único objeto: una *solución*, que es una lista de rutas junto con su costo, la carga de cada ruta y una bandera que indica si es factible. Cada componente recibe una solución y entrega otra, normalmente más corta; algunos pueden pasar por soluciones infactibles durante su trabajo, pero lo que entregan al final es factible. La Figura 2 muestra ese recorrido.

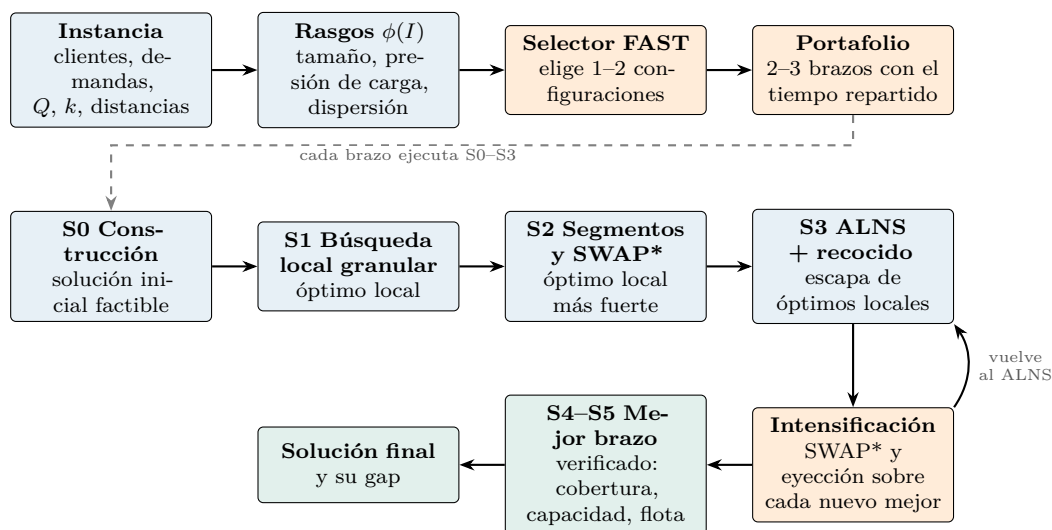


Figura 2: Recorrido de la información en SENDa. Los rasgos de la instancia deciden cuántos brazos se ejecutan y con qué configuración. Cada brazo recorre la cadena S0–S3 con su parte del reloj; la intensificación se activa cada vez que el ALNS encuentra un nuevo mejor global. Al final se verifica cada brazo y se conserva el mejor.

El orden no es arbitrario. Cada etapa corrige un tipo de defecto que la anterior no puede ver:

1. La **construcción** produce rápidamente rutas factibles, pero con cruces y desvíos evidentes.
2. La **búsqueda local granular** elimina esos defectos con movimientos pequeños hasta que ningún cambio pequeño mejora: la solución queda en un *óptimo local*.
3. Los **segmentos y SWAP*** amplían lo que cuenta como “movimiento pequeño” y permiten salir de óptimos locales que los movimientos básicos no pueden romper.
4. El **ALNS** destruye y reconstruye partes grandes de la solución, lo que le permite cambiar su estructura: qué clientes comparten ruta. Así escapa de óptimos locales profundos.
5. La **intensificación** pule a fondo cada nueva mejor solución.

6. Los **brazos del portafolio** repiten el proceso con configuraciones distintas y conservan el mejor resultado, lo que reduce el riesgo de que una sola trayectoria se estanque.

5. Los componentes en detalle

Cada componente se presenta con las mismas preguntas: qué es, cómo funciona, un ejemplo, por qué es importante y cuánto aporta. Las cifras de aporte provienen de la ablación de la Sección 9.3 (lo que se pierde al retirarlo) y de la cascada de la Sección 6 (lo que se gana al agregarlo).

5.1. Construcción inicial

Qué es. Un procedimiento rápido que produce una primera solución completa.

Cómo funciona. Si la flota es fija se usa *best-fit-decreasing* (BFD): los clientes se ordenan de mayor a menor demanda y cada uno se asigna al vehículo en el que, después de cargarlo, quedaría el menor espacio libre. Si en ningún vehículo cabe, se asigna al menos cargado y la solución queda temporalmente infactible; las etapas siguientes la reparan. Dentro de cada vehículo, el cliente se inserta en la posición que menos alarga la ruta. Si la flota es libre se usa *vecino más cercano*: desde el depósito se visita al cliente más próximo que todavía cabe y, al llenarse el vehículo, se abre una ruta nueva.

Ejemplo resuelto: best-fit-decreasing

Sea $Q = 10$, $k = 2$ y demandas 7, 5, 4, 3, 1. El cliente de demanda 7 va al vehículo A (queda libre 3). El de 5 no cabe en A y va a B (queda libre 5). El de 4 cabe solo en B (queda libre 1). El de 3 cabe en A, que queda lleno, y el de 1 cabe en B, que queda lleno. Resultado: $A = \{7, 3\}$ y $B = \{5, 4, 1\}$, ambos con carga 10.

Por qué importa. Con flota fija, lo primero es repartir la demanda entre los k vehículos sin exceder su capacidad, y en muchas instancias A/B/E la demanda total casi llena la flota. Colocar primero a los clientes grandes, cuando todavía hay espacio, aumenta la probabilidad de lograr un reparto factible. La construcción no intenta que las rutas sean cortas: de eso se encargan las etapas siguientes.

Cuánto aporta. Es el punto de partida (etapa S0 de la cascada): en promedio deja un gap de 81.008 % en A/B/E y 25.786 % en X.

5.2. Búsqueda local granular

Qué es. Un procedimiento que prueba cambios pequeños sobre la solución y aplica los que reducen la distancia, hasta que ninguno la mejora.

Cómo funciona. Se usan cinco familias de movimientos: *relocate* (mover un cliente a otra posición), *swap* (intercambiar dos clientes), *2-opt* (invertir un tramo de una ruta para eliminar un cruce), *2-opt** (intercambiar las colas de dos rutas) y *cadena de eyección* (mover un cliente que a su vez desplaza a otro). Probar todos los pares de clientes cuesta $O(n^2)$ por pasada, así que se aplica la *restricción granular* de Toth y Vigo [13]: cada cliente solo considera a sus $k = 15$ vecinos más cercanos, porque casi nunca conviene unir en una ruta a dos clientes muy alejados. Además, cada movimiento se evalúa con una fórmula de costo constante, sin reconstruir la ruta. Para mover al cliente u , cuyos vecinos en su ruta son p_u y n_u , entre los nodos x e y de otra ruta:

$$\Delta = \underbrace{d(x, u) + d(u, y) - d(x, y)}_{\text{costo de insertarlo}} - \underbrace{[d(p_u, u) + d(u, n_u) - d(p_u, n_u)]}_{\text{ahorro de sacarlo}}. \quad (3)$$

Si $\Delta < 0$ y la ruta destino tiene capacidad, el movimiento se aplica.

Ejemplo resuelto: delta de un relocate

Sean $d(p_u, u) = 4$, $d(u, n_u) = 5$, $d(p_u, n_u) = 7$, $d(x, u) = 3$, $d(u, y) = 2$ y $d(x, y) = 4$. Sacar a u ahorra $4 + 5 - 7 = 2$ y meterlo entre x e y cuesta $3 + 2 - 4 = 1$. Entonces $\Delta = 1 - 2 = -1$: el movimiento acorta la solución en una unidad. El cálculo requirió seis consultas a la matriz de distancias, sin recorrer ninguna ruta.

Por qué importa. Es la herramienta que hace el trabajo fino y la que más veces se ejecuta: la usan la etapa S1, cada candidato del ALNS y la intensificación. La restricción granular y la evaluación en tiempo constante le permiten revisar muchos más movimientos en el mismo tiempo que una búsqueda sobre todos los pares de clientes.

Capacidad durante el pulido. El pulido inicial trabaja con un costo *penalizado*, $C(S) + \lambda \cdot \text{exceso}(S)$: tolera excesos de capacidad si ahorran suficiente distancia. Si al terminar queda algún exceso, repite la búsqueda con una penalización diez veces mayor para recuperar la factibilidad. Esa segunda búsqueda no siempre lo logra: cuando la carga está muy ajustada, el pulido puede terminar con un pequeño exceso (Sección 6), y la factibilidad se recupera más adelante, dentro del ALNS.

Cuánto aporta. Retirar la restricción granular —es decir, usar el vecindario completo— empeora el gap en X en 0.267 pp: con el mismo reloj, el vecindario completo evalúa menos iteraciones útiles.

5.3. ALNS con recocido simulado

Qué es. ALNS (*Adaptive Large Neighborhood Search*, búsqueda adaptativa de vecindarios grandes [12, 11]) es el motor principal. En cada iteración *destruye* una parte de la solución —retira entre el 10 y el 35 % de los clientes— y la *repara* reinsertándolos.

Por qué hace falta. La búsqueda local se detiene en un óptimo local: ninguna modificación pequeña mejora. Pero la mejor solución puede requerir un cambio grande, por ejemplo que diez clientes cambien de ruta a la vez. El ALNS provoca esos cambios grandes de forma controlada.

Cómo destruye. Hay cuatro operadores, cada uno con una intuición distinta:

- *aleatorio*: retira clientes al azar para diversificar;
- *peor contribución*: retira, con sesgo aleatorio, los clientes que más alargan su ruta, es decir, los que probablemente están mal ubicados;
- *Shaw* [12]: retira un grupo de clientes cercanos entre sí, para que se redistribuyan juntos;
- *eliminación de rutas*: retira rutas completas. Es el operador pensado para reducir el número de rutas; por eso, cuando la solución excede la flota, se elige con probabilidad 0.8.

Cómo repara. La reparación *voraz* inserta cada cliente en su posición más barata. La reparación *regret-2* inserta primero al cliente que más perdería si se le deja para después.

Ejemplo resuelto: reparación regret-2

Quedan por insertar a y b . La mejor inserción de a cuesta 5 y la segunda 12, así que su “arrepentimiento” es $12 - 5 = 7$. La mejor de b cuesta 3 y la segunda 4: arrepentimiento 1. Aunque b es más barato, regret-2 inserta primero a a : si otro cliente ocupa su mejor lugar, a

tendría que pagar 12, mientras que b solo perdería una unidad.

Capacidad durante la reparación. En los brazos estándar, la reparación asigna una penalización enorme (10^9) a cualquier exceso de capacidad, así que en la práctica solo inserta donde cabe. El brazo penalizado del portafolio (Sección 5.8) relaja esta regla.

Cómo aprende. Cada operador tiene un peso. Se elige por ruleta, con probabilidad proporcional a su peso, y después se actualiza $w \leftarrow 0.99w + 0.01\sigma$, con premio $\sigma = 5$ si produjo un nuevo mejor global, 2 si mejoró la solución actual, 0.5 si solo fue aceptado y 0 si fue rechazado. Así, los operadores que funcionan en la instancia actual se usan más.

Cómo decide si acepta. El candidato reparado se pule con búsqueda local y se acepta según recocido simulado [8]: si mejora, se acepta siempre; si empeora en Δ , se acepta con probabilidad $e^{-\Delta/T}$. La temperatura baja de forma geométrica, $T(f) = T_0(T_f/T_0)^f$, donde $f \in [0, 1]$ es la fracción del presupuesto consumido, $T_0 = 0.02C$ y $T_f = 0.0002C$, con C el costo de la mejor solución inicial.

Ejemplo resuelto: aceptar una solución peor

Si el candidato es 10 unidades más largo ($\Delta = 10$) y la temperatura es $T = 20$, se acepta con probabilidad $e^{-10/20} = 0.61$. Al final de la búsqueda, con $T = 1$, la misma decisión tendría probabilidad $e^{-10} \approx 0.00005$. El algoritmo explora mucho al principio y se vuelve exigente al final.

Algoritmo 1 Núcleo ALNS de un brazo de SENDA

Entrada: solución S (ya pulida), presupuesto B en segundos o iteraciones

Salida: mejor solución factible S^*

```

1:  $S^* \leftarrow S$ ; pesos  $w \leftarrow \mathbf{1}$ 
2: mientras no se agote  $B$  y no haya 5 000 iteraciones seguidas sin mejora hacer
3:   elegir destrucción  $\delta$  y reparación  $\rho$  por ruleta con pesos  $w$ 
4:    $S' \leftarrow \rho(\delta(S))$ ; pulir  $S'$  con búsqueda local granular
5:   si  $S'$  es factible y  $C(S') < C(S^*)$  entonces
6:     intensificar  $S'$  (SWAP* y eyección);  $S^* \leftarrow S'$ ;  $S \leftarrow S'$ ;  $\sigma \leftarrow 5$ 
7:   si no, si  $\Delta = C(S') - C(S) < 0$  o  $u \sim U(0, 1) < e^{-\Delta/T(f)}$  entonces
8:      $S \leftarrow S'$ ;  $\sigma \leftarrow 2$  si  $\Delta < 0$ , si no 0.5
9:   si no
10:     $\sigma \leftarrow 0$ 
11:  fin si
12:   $w_\delta \leftarrow 0.99w_\delta + 0.01\sigma$ ;  $w_\rho \leftarrow 0.99w_\rho + 0.01\sigma$ 
13: fin mientras
14: devolver  $S^*$ 

```

Por qué importa y cuánto aporta. Es el componente dominante. En la ablación, retirar el ALNS —dejando solo construcción y pulido— eleva el gap medio en X de 4.285 % a 16.702 %: una pérdida de 12.4 puntos. En la cascada (Sección 6), el ALNS es la primera etapa que entrega siempre soluciones factibles, y en A/B/E baja el gap de las soluciones factibles de 3.475 % a 0.319 %.

5.4. Movimientos de segmento y Or-opt-3

Qué es. Una extensión de la búsqueda local: en lugar de mover un cliente, se mueve un *segmento* de dos o tres clientes consecutivos (*relocate-2*, *Or-opt-3*), o se intercambia un par de clientes por uno solo (*swap-2*).

Por qué importa. Con frecuencia, dos clientes consecutivos están bien colocados uno junto al otro, pero el par completo estaría mejor en otra ruta. Moverlos por separado exige dos pasos, y el primero casi siempre empeora la solución, así que la búsqueda local no lo da. Moverlos juntos convierte esa mejora en un solo paso que sí se detecta.

Cuánto aporta. Retirarlos eleva el gap en X en 0.528 pp; en 36 de 41 instancias la versión completa es mejor. Es el segundo componente más importante después del ALNS.

5.5. SWAP*

Qué es. Un movimiento entre dos rutas propuesto por Vidal para HGS-CVRP [15]. El *swap* clásico intercambia al cliente u de la ruta A con el cliente v de la ruta B, y cada uno ocupa la posición exacta del otro. SWAP* también los intercambia, pero coloca a cada uno en su *mejor* posición dentro de la otra ruta.

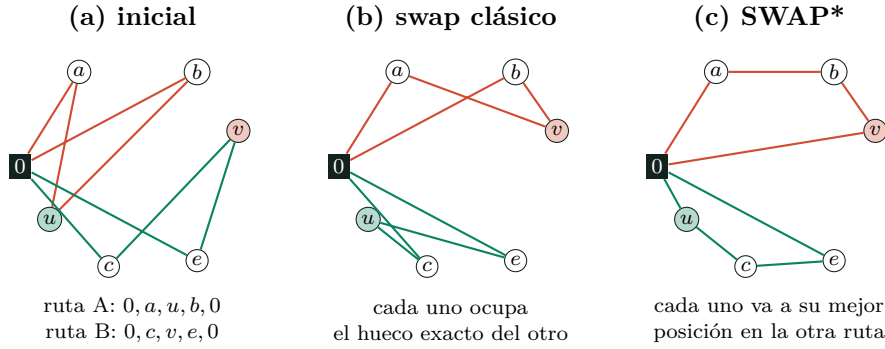


Figura 3: Diferencia entre el swap clásico y SWAP*. Los puntos no se mueven; solo cambian las rutas. En (a), u está mal ubicado en la ruta roja y v en la verde. El swap clásico (b) los intercambia, pero cada uno queda en el hueco que dejó el otro, lejos de donde conviene. SWAP* (c) los intercambia y coloca a cada uno en su mejor posición, así que encuentra mejoras que el swap clásico descarta.

Cómo funciona. El cambio de costo es

$$\Delta_{\text{SWAP}^*}(u, v) = \text{ins}_{B \setminus v}^*(u) + \text{ins}_{A \setminus u}^*(v) - \text{rem}_A(u) - \text{rem}_B(v), \quad (4)$$

donde rem es lo que se ahorra al retirar al cliente de su ruta e ins^* el costo de su mejor inserción en la otra ruta. Para hacerlo rápido, para cada cliente y cada ruta se guardan de antemano las tres mejores posiciones de inserción; así, el mejor lugar se obtiene en tiempo constante aunque el cliente saliente ocupe una de ellas. La versión original elige qué pares de rutas comparar mediante sectores polares, lo que requiere coordenadas; la nuestra usa el grafo granular y funciona también con matrices de distancia sin coordenadas.

Ejemplo resuelto: por qué SWAP* encuentra más

Retirar u de A ahorra 10 y retirar v de B ahorra 9. En el swap clásico, u en el hueco de v cuesta 12 y v en el hueco de u cuesta 11: $\Delta = 12 + 11 - 10 - 9 = +4$, así que se rechaza. En SWAP*, la mejor posición de u en B cuesta 7 y la de v en A cuesta 8: $\Delta = 7 + 8 - 10 - 9 = -4$, y el intercambio mejora la solución en 4 unidades.

Por qué importa y cuánto aporta. Explora intercambios entre rutas mucho más ricos al mismo costo por evaluación. Retirarlo eleva el gap en X en 0.413 pp, con 38 de 41 instancias a favor de la versión completa. Se aplica en el pulido inicial y en la intensificación, pero no sobre cada candidato del ALNS, porque eso reduciría demasiado el número de iteraciones.

5.6. Intensificación del mejor global

Cada vez que el ALNS encuentra una nueva mejor solución factible, se pule a fondo: búsqueda local granular, hasta dos pasadas de SWAP* y cadenas de eyección, con un máximo de 50 pulidos por brazo. Es el momento en que SWAP* y las cadenas de eyección trabajan: se reservan para las soluciones que ya demostraron ser prometedoras.

5.7. Selector FAST

Qué es. Una regla congelada que, a partir de rasgos baratos de calcular, decide qué configuración adicional ejecutar además de la base.

Cómo funciona. Calcula el número de clientes n , la flota k , la presión de carga $\sum q_i/(kQ)$, los coeficientes de variación de la demanda y de la distancia al depósito, la concentración angular de los clientes alrededor del depósito y su densidad. Con esos valores recorre una lista de condiciones (Apéndice B) y la primera que se cumple elige una de las configuraciones de la Tabla 1: vecindarios más amplios, destrucción más agresiva o más iteraciones.

Tabla 1: Configuraciones que el selector puede elegir. Todas usan segmentos y SWAP* con dos pasadas.

Configuración	Idea	vecinos k	destrucción
baseline_cap2_calls50	rama base, siempre se ejecuta	15	10–35 %
granular_k25 / granular_k40	vecindario más amplio	25 / 40	10–35 %
large_destroy_k25	destrucción más agresiva	25	15–45 %
medium_destroy_k25	destrucción moderada	25	10–25 %
long_10k_k25	hasta 10 000 iteraciones	25	10–35 %

Por qué importa y cuánto aporta. No hay una configuración ideal para todas las instancias: con demanda muy variable conviene destruir más; con clientes concentrados, ampliar el vecindario. Sustituir la rama elegida por un reinicio de la rama base eleva el gap en X en 0.145 pp.

5.8. El portafolio: la configuración evaluada

Qué es. Es la configuración congelada de SENDA que se evaluó en A/B/E. Su definición operativa:

1. un brazo con la configuración base y otro con la configuración que elige FAST, si es distinta;
2. Or-opt-3 activo y corte por estancamiento: un brazo se detiene si pasa 5 000 iteraciones sin mejorar;
3. un **brazo penalizado** adicional con la última configuración.

Cada brazo recibe una parte igual del reloj (80 % del presupuesto dividido entre 2 o 3 brazos) y se conserva el mejor resultado verificado. En X se usó la variante FAST-PEN4, que difiere en que sus brazos estándar no usan Or-opt-3 ni corte por estancamiento.

El brazo penalizado. En los brazos estándar, la reparación del ALNS trata la capacidad como una restricción dura: nunca acepta un exceso. El brazo penalizado permite atravesar soluciones con exceso de capacidad, pagando una penalización proporcional:

$$C_\lambda(S) = C(S) + \lambda \cdot \sum_{r \in S} \max\left(0, \sum_{i \in r} q_i - Q\right). \quad (5)$$

El peso λ se ajusta automáticamente cada 50 iteraciones: si menos del 25 % de los candidatos son factibles, λ crece un 25 %; si más del 45 % lo son, decrece un 15 %. Así, la búsqueda se mantiene cerca de la frontera entre soluciones factibles e infactibles, que es donde suelen estar las mejores.

Idea clave: por qué ayuda la penalización

Cuando la capacidad está muy ajustada, las soluciones factibles forman islas separadas: para pasar de una buena a otra mejor hay que atravesar soluciones con exceso. La penalización construye ese puente. Como solo cuenta la mejor solución *factible* de cada brazo, el brazo penalizado nunca entrega una solución inválida. Su costo es otro: consume su parte del reloj, que en el peor caso no aporta nada.

Cuánto aporta. Sustituir el brazo penalizado por un reinicio normal con el mismo tiempo eleva el gap en X en 0.145 pp (23 instancias a favor y 18 en contra; $p_H = 0.048$, la evidencia más débil de la ablación). En una comparación a cómputo igual, añadirlo reduce el gap en X de 3.678 % a 3.461 % (31 de 41 instancias, $p = 4.3 \times 10^{-5}$).

5.9. Parámetros

Tabla 2: Parámetros del solver. Ninguno se ajusta por instancia.

Componente	Parámetro	Valor
Granular	vecinos por cliente (completo si $n \leq 60$)	15
ALNS	fracción destruida	$U(0.10, 0.35)$ según rama
ALNS	decaimiento de pesos; premios	0.99; 5 / 2 / 0.5 / 0
Recocido	T_0, T_f (fracción del costo)	0.02, 0.0002
Flota	probabilidad de forzar eliminación de rutas	0.8
Intensificación	pasadas SWAP*; pulidos por brazo	2; 50
Portafolio	corte por estancamiento; reserva de reloj	5 000 iteraciones; 20 %
Brazo penalizado	ajuste de λ cada 50 iteraciones	$\times 1.25$ / $\times 0.85$

6. Cómo se acumula la calidad: el experimento de cascada

La ablación responde “¿cuánto se pierde si quito un componente?”. La cascada responde la pregunta complementaria: “¿qué calidad tengo después de cada etapa?”. Se ejecutaron seis etapas con las mismas 10 semillas del protocolo, en las 63 instancias A/B/E (flota fija) y las 41 X (flota libre). Se usó el binario de la ablación de componentes, porque es el único que permite detener el solver antes del ALNS; su versión completa obtuvo en X un gap de 4.285 %, cercano pero no idéntico al 4.119 % del binario confirmatorio. Las etapas son:

Etapas	Solución disponible
S0	construcción, sin ninguna mejora
S1	construcción + búsqueda local granular básica
S2	construcción + búsqueda local con segmentos, Or-opt-3 y SWAP*
S3	primer brazo del portafolio completo (con ALNS), con su parte del reloj
S4	mejor de S3 y el brazo de la configuración elegida por FAST
S5	portafolio completo: se agrega el brazo penalizado

S1 y S2 son dos pulidos independientes de la misma construcción: S2 no parte de S1, sino que usa un conjunto de movimientos más rico. Por eso, en una réplica concreta, S2 puede quedar peor que S1, aunque en promedio sea mejor. Del mismo modo, cada brazo de S3–S5 hace su propia construcción, su pulido y su ALNS.

Para cada etapa se reportan tres medidas, porque una sola engaña:

- el **porcentaje de soluciones factibles** que entrega la etapa;
- el **gap de las soluciones factibles**, promediado primero por instancia (entre paréntesis, cuántas instancias tuvieron al menos una réplica factible);
- el **gap disponible**, la regla fijada antes de ver los resultados: si la etapa entrega una solución infactible, la solución que realmente se tiene en la mano sigue siendo la construcción.

Tabla 3: Calidad tras cada etapa (10 réplicas por instancia). “Fact.” es el porcentaje de soluciones factibles; “gap factibles” promedia solo las soluciones factibles (entre paréntesis, el número de instancias con al menos una); “gap disponible” sustituye cada solución infactible por la construcción.

Etapa	A/B/E (63 inst., 2 s, flota fija)			X (41 inst., 4 s)		
	fact.	gap factibles	gap disponible	fact.	gap factibles	gap disponible
S0 construcción	100 %	81.008 % (63)	81.008 %	100 %	25.786 % (41)	25.786 %
S1 + local granular	83 %	4.671 % (62)	18.972 %	71 %	6.262 % (39)	14.453 %
S2 + segmentos y SWAP*	49 %	3.475 % (51)	46.181 %	42 %	5.244 % (32)	19.508 %
S3 + ALNS (brazo base)	100 %	0.319 % (63)	0.319 %	100 %	4.971 % (41)	4.971 %
S4 + rama FAST	100 %	0.243 % (63)	0.243 %	100 %	4.459 % (41)	4.459 %
S5 + brazo penalizado	100 %	0.115 % (63)	0.115 %	100 %	4.121 % (41)	4.121 %

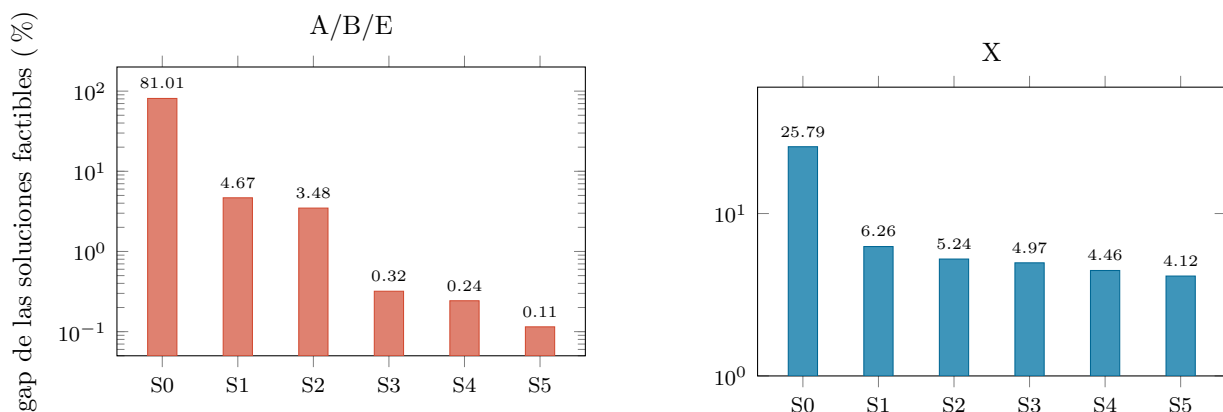


Figura 4: Gap de las soluciones factibles tras cada etapa (escala logarítmica). En A/B/E la caída decisiva ocurre al entrar el ALNS (S3); en X la búsqueda local ya hace gran parte del trabajo y los brazos adicionales (S4, S5) aportan más que en A/B/E.

Ejemplo resuelto: una solución en tránsito

En la instancia A-n80-k10 (BKS 1763), réplica 0, el costo evoluciona así: construcción 3180 (80.37 %); búsqueda local 1827 (3.63 %); segmentos y SWAP* 1865 (5.79 %); primer brazo con ALNS 1783 (1.13 %); con la rama FAST 1766 (0.17 %); y con el brazo penalizado 1766 (0.17 %).

Lectura.

1. **La construcción no busca rutas cortas.** Best-fit-decreasing reparte la demanda sin mirar la geografía, y por eso en A/B/E deja gaps cercanos al 80 %. En X, el vecino más cercano sí mira distancias y parte de un gap menor.
2. **La búsqueda local hace la mayor reducción en distancia.** Con solo el pulido, el gap de las soluciones factibles baja a 4.671 % en A/B/E y a 6.262 % en X.
3. **Segmentos y SWAP* acortan más, pero rompen la capacidad.** Las soluciones factibles de S2 son mejores que las de S1, pero solo el 49 % de los pulidos de S2 terminó factible en A/B/E (42 % en X). El pulido penalizado cruza a la zona infactible para ahorrar distancia y no siempre logra regresar. Por eso el gap disponible de S2 es malo: en la mitad de los casos no hay más que la construcción.
4. **El ALNS es la primera etapa que entrega, a la vez, calidad y factibilidad en el 100 % de los ensayos.** En A/B/E baja el gap a 0.319 %, con una mediana cercana a cero. Su reparación y su búsqueda local tratan la capacidad como restricción dura en cada iteración.
5. **Los brazos adicionales afinan.** En A/B/E, la rama FAST y el brazo penalizado reducen el gap de 0.319 % a 0.115 %, y el número de instancias que alcanzan la BKS en todas sus réplicas pasa de 31 a 37. En X, donde un solo brazo tiene poco tiempo, su aporte relativo es mayor: de 4.971 % a 4.121 %.

Lección metodológica: una solución inválida siempre parece mejor

Si se promediara el costo de todas las soluciones de S2, incluidas las infactibles, su gap sería 0.238 % en A/B/E: *mejor* que el del ALNS (0.319 %). Es la misma trampa del caso de estudio de la Sección 10: violar una restricción —capacidad o flota— casi siempre permite rutas más cortas. Por eso, toda comparación debe contar solo soluciones verificadas.

Lección metodológica: cascada y ablación no dicen lo mismo

En la cascada, lo que se atribuye a cada etapa depende del orden en que se agregan: la búsqueda local parece hacer la mayor parte del trabajo porque va primero y encuentra una solución muy mejorable. En la ablación, retirar un componente mide lo que los demás *no* pueden compensar: sin el ALNS, el gap en X sube 12.4 puntos, porque la búsqueda local sola no alcanza las soluciones que el ALNS sí encuentra. Ambas vistas son necesarias. Por ejemplo, la cascada no muestra el valor de la restricción granular, porque no es una etapa sino la forma en que se ejecutan todas; solo la ablación lo revela.

7. Complejidad computacional: cuánto cuesta cada paso

Las secciones anteriores explicaron qué hace cada componente y cuánta calidad aporta. Falta una pregunta igual de importante: *cuánto cuesta*. En SENDA el presupuesto es un reloj fijo, así que el costo de cada paso decide cuántas iteraciones caben en ese reloj y, con ello, hasta dónde llega la búsqueda. Esta sección introduce la herramienta para razonar sobre ese costo, la notación O grande, la aplica a cada componente y comprueba la predicción con una medición.

7.1. La notación O grande

Contar segundos no sirve para comparar algoritmos: el mismo programa tarda distinto en cada computadora. Lo que sí es estable es *cómo crece* el número de operaciones cuando crece el tamaño de la entrada n . La notación O grande describe ese crecimiento. Se dice que un algoritmo

es $O(g(n))$ si, a partir de cierto tamaño, su número de operaciones nunca supera un múltiplo fijo de $g(n)$. Formalmente, $f(n) = O(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0. \quad (6)$$

En la práctica bastan tres reglas:

- **Se descartan constantes y términos menores.** $3n^2 + 50n + 7$ es $O(n^2)$: para n grande, el término n^2 domina a los demás.
- **Los pasos en secuencia se suman y domina el mayor.** Ordenar ($O(n \log n)$) y después recorrer la lista ($O(n)$) cuesta $O(n \log n)$.
- **Los bucles anidados se multiplican.** Recorrer cada cliente y, para cada uno, a todos los demás cuesta $O(n \cdot n) = O(n^2)$.

Tabla 4: Cómo crecen las funciones de complejidad más comunes. Los logaritmos son en base 2.

Función	$n = 100$	$n = 1\,000$	Ejemplo en esta guía
$O(1)$	1	1	evaluar el delta de un movimiento (Ec. 3)
$O(\log n)$	6.6	10	—
$O(n)$	100	1 000	calcular el costo de una solución
$O(n \log n)$	664	9 966	ordenar los clientes por demanda
$O(n^2)$	10^4	10^6	calcular la matriz de distancias
$O(n^3)$	10^6	10^9	reparación regret-2 con rutas de longitud fija
$O(2^n)$	1.3×10^{30}	1.1×10^{301}	probar todos los subconjuntos de clientes
$O(n!)$	9.3×10^{157}	4.0×10^{2567}	probar todos los órdenes de visita

Idea clave: O grande describe el crecimiento, no el tiempo

Dos algoritmos $O(n)$ pueden tardar muy distinto: la notación oculta las constantes. Lo que sí dice es qué pasa al crecer la instancia. Si un paso es $O(n^2)$, duplicar el número de clientes multiplica su costo por cerca de 4; si es $O(n^3)$, por cerca de 8.

7.2. Por qué el CVRP no se resuelve exactamente

El CVRP es *NP-difícil* [9]: contiene como caso particular al problema del agente viajero (un solo vehículo de capacidad suficiente), y no se conoce ningún algoritmo que lo resuelva de forma exacta en tiempo polinomial, es decir, en $O(n^c)$ para alguna constante c . Una búsqueda exhaustiva tendría que revisar, solo para el orden de visita, del orden de $n!$ posibilidades, y además todas las formas de repartir a los clientes entre vehículos.

Ejemplo resuelto: el tamaño del espacio de búsqueda

Con un solo vehículo y $n = 20$ clientes hay $20! \approx 2.4 \times 10^{18}$ órdenes de visita. Una computadora que evaluara mil millones de órdenes por segundo tardaría $2.4 \times 10^{18} / 10^9 \approx 2.4 \times 10^9$ segundos: unos 77 años. Con 21 clientes, 21 veces más.

Los métodos exactos modernos certifican óptimos en muchas instancias de referencia [10], pero su tiempo crece de forma difícil de predecir. Una heurística cambia la garantía de optimalidad por algo que sí se puede controlar: cada iteración cuesta un número *polinomial* de operaciones, y el número de iteraciones lo fija el reloj.

7.3. Notación para el análisis

Además de n (número de clientes), el costo de SENDA depende de cuatro cantidades:

- k : vecinos por cliente en la restricción granular (15, 25 o 40 según la rama; todos los demás clientes si $n \leq 60$);
- R : número de rutas;
- $L \approx n/R$: longitud media de una ruta, en clientes;
- q : clientes retirados por una destrucción del ALNS, $q = \theta n$ con θ entre 0.10 y 0.35 (Tabla 2).

Las rutas se guardan como arreglos de clientes. Insertar o retirar un cliente de un arreglo exige desplazar a los que le siguen, así que cuesta $O(L)$.

7.4. Costo de cada componente

La Tabla 5 resume el análisis del código de `native/send_core.cpp`. Las cifras son de peor caso por llamada.

Tabla 5: Complejidad de cada componente de SENDA.

Componente	Costo	De dónde sale
<i>Una vez por corrida</i>		
Matriz de distancias	$O(n^2)$	una distancia por par de nodos; 4 bytes por par (4 MB con 1 000 clientes)
Listas de vecinos granulares	$O(n^2 \log n)$	para cada cliente se ordenan las distancias a los demás
Construcción <i>best-fit-decreasing</i>	$O(n \log n + nR + nL)$	ordenar; elegir vehículo; insertar en la mejor posición
Construcción vecino más cercano	$O(n^2)$	en cada paso se busca al cliente más próximo entre los restantes
<i>Búsqueda local</i>		
Evaluar un movimiento	$O(1)$	seis a ocho consultas a la matriz (Ec. 3)
Aplicar un movimiento	$O(L)$	insertar o retirar en el arreglo de la ruta y actualizar sus cargas
Una pasada granular	$O(nk)$ evaluaciones	cada cliente prueba sus k vecinos con cada familia de movimientos
<i>Una iteración del ALNS</i>		
Destrucción aleatoria	$O(n)$	barajar y retirar
Destrucción por peor contribución	$O(n \log n + qn)$	ordenar por contribución; cada retiro reacomoda la lista
Destrucción Shaw	$O(qn \log n)$	antes de cada retiro se reordena la lista por cercanía
Destrucción de rutas	$O(n + R \log R)$	elegir rutas y separar sus clientes
Reparación voraz	$O(q(n + k + R))$	por cliente: reindexar la solución y revisar sus opciones de inserción
Reparación regret-2	$O(qn + q^2(k + R) \log(k + R))$	en cada ronda se recalculan y ordenan las opciones de <i>todos</i> los clientes pendientes
<i>Intensificación</i>		
SWAP* (una pasada)	$O(PL^2)$	$P \leq \min(R^2, nk)$ pares de rutas vecinas; en cada par, las tres mejores inserciones de cada cliente en la otra ruta
Cadenas de eyección (una pasada)	$O(nRL^3) = O(n^2L^2)$	cada cliente contra cada ruta llena, cada par de clientes a expulsar y una reinserción por combinación
Eliminar una ruta sobrante	$O(n^2)$	cada ruta candidata reinserta sus clientes en todas las posiciones de las demás

Ejemplo resuelto: por qué la restricción granular cambia el orden de magnitud

Con $n = 400$, una pasada que probara todos los pares de clientes haría unas $400^2 = 160\,000$ evaluaciones por familia de movimientos. Con $k = 15$ hace $400 \times 15 = 6\,000$: 27 veces menos. Como cada evaluación es $O(1)$, la pasada completa pasa de $O(n^2)$ a $O(nk)$, que es lineal en n porque k es fijo.

El costo de una iteración del ALNS. Una iteración suma una destrucción, una reparación, la búsqueda local del candidato y el cálculo de su costo ($O(n)$). La búsqueda local tiene un tope de tiempo (0.08 s por candidato), así que no crece sin límite. Con $q = \theta n$, los términos que dominan son la destrucción Shaw, $O(n^2 \log n)$, y la reparación regret-2, $O(n^2(k + R) \log(k + R))$. Si la longitud de las rutas es fija, R crece con n y regret-2 se acerca a $O(n^3)$. La predicción es, entonces, que una iteración se encarece entre 4 y 8 veces cada vez que se duplica n .

Por qué SWAP* y las cadenas de eyección se reservan. Una pasada de cadenas de eyección es $O(n^2 L^2)$: con 400 clientes en rutas de 12 hay del orden de un millón de combinaciones (cliente entrante, ruta destino y par de clientes expulsados), cada una con reinserciones de costo $O(L)$, unos 2×10^7 pasos por pasada. Aplicarla a cada candidato del ALNS consumiría el reloj en pocas iteraciones. Por eso SWAP* y las cadenas solo se aplican al pulido inicial y a cada nuevo mejor global (Sección 5.5): son operadores caros que se usan donde más rinden.

7.5. Comprobación experimental

Para contrastar el análisis con el programa real se midió cuántas iteraciones del ALNS completa SENDA en un reloj fijo, en instancias aleatorias de 100 a 800 clientes. La Tabla 6 y la Figura 5 muestran el resultado.

Tabla 6: Iteraciones del ALNS en 4 s (un hilo, mediana de 3 semillas) y tiempo medio por iteración.

n	Iteraciones	ms por iteración	Factor al duplicar n
100	11 073	0.32	—
200	2 714	1.30	$\times 4.1$
400	703	5.01	$\times 3.9$
800	99	35.56	$\times 7.1$

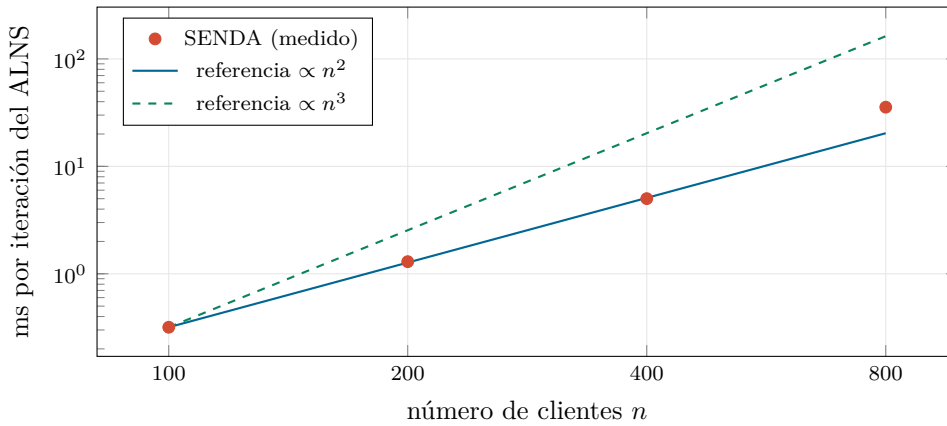


Figura 5: Tiempo por iteración frente a n en escala logarítmica. En esta escala, una función cn^a es una recta de pendiente a . Hasta 400 clientes los puntos siguen la referencia n^2 ; de 400 a 800 la pendiente se acerca a la de n^3 .

La medición confirma el análisis. Entre 100 y 400 clientes, cada duplicación de n multiplica el costo de una iteración por cerca de 4, como corresponde a un costo $O(n^2)$. De 400 a 800 el factor sube a 7.1, más cerca de n^3 : es lo esperado cuando el término q^2R de regret-2 empieza a pesar, aunque en ese rango también influye la intensificación, porque cada pulido del mejor global cuesta más y hay menos iteraciones entre las que repartirlo. La medición no separa ambos efectos.

Lección metodológica: con reloj fijo, la complejidad se convierte en calidad

Con un costo por iteración de orden n^2 , el número de iteraciones que caben en el reloj cae como $1/n^2$: de 100 a 400 clientes, de 11 073 a 703 iteraciones, 16 veces menos. Es consistente con lo observado en las instancias X (Sección 9): con 199 a 428 clientes y poco más de un segundo por brazo, el ALNS dispone del orden de cientos de iteraciones y apenas mejora lo que ya logró la búsqueda local. Para que una heurística escale no basta con que cada paso sea polinomial; el exponente importa.

Para reproducir

Binario compilado de `native/senda_core.cpp` (SHA-256 `d63db4bc6436...`) con `python -m lattimex build`; llamada directa a `senda_solve` con 4s de reloj, flota libre, $k = 15$, movimientos de segmento y SWAP* activos (`use_segment_moves=3`), sin brazo penalizado y semillas 1, 2 y 3. Instancias: depósito y clientes uniformes en $[0, 1000]^2$, demandas enteras uniformes entre 1 y 10, capacidad 66 (unos 12 clientes por ruta). Equipo: Intel Core i5-8250U. Las cifras absolutas cambian con el equipo; los factores al duplicar n son los que deben reproducirse.

7.6. Dónde podría bajar el costo

El análisis también señala mejoras posibles. Ninguna está aplicada en el motor; se proponen como ejercicios.

1. **Destrucción Shaw sin reordenar.** En lugar de ordenar toda la lista antes de cada retiro ($O(n \log n)$), elegir entre los vecinos granulares del cliente de referencia, ya ordenados: $O(k)$ por retiro.
2. **Regret-2 incremental.** Al insertar un cliente solo cambian las opciones que tocan la ruta modificada. Guardar las dos mejores opciones de cada cliente pendiente y recalcular solo las afectadas evita repetir el trabajo de todos los pendientes en cada ronda.
3. **Vecinos con selección parcial.** Para obtener los k más cercanos no hace falta ordenar a todos: `std::nth_element` los separa en $O(n)$ por cliente, y la construcción de las listas baja de $O(n^2 \log n)$ a $O(n^2)$.

8. Cómo evaluar una heurística sin engañarse

Una heurística es aleatoria y su calidad depende del tiempo que se le dé. Por eso, la forma de medirla importa tanto como el algoritmo. Este proyecto siguió seis reglas.

1. **El mismo reloj para todos.** El perfil principal da a cada ensayo 2s de tiempo real en un solo hilo en A/B/E y 4s en X, con una tolerancia del 10 %. Un ensayo que excede la tolerancia se descarta. SENDA reserva el 20 % del límite para leer la instancia y verificar la solución, así que su núcleo usa el 80 % restante.

2. Réplicas, no el mejor de diez. Cada instancia se resuelve 10 veces con semillas distintas y se reporta la *media*. Elegir la mejor de diez corridas exagera la calidad que un usuario obtendría en una sola ejecución.

3. Verificación independiente y uniforme. Cada solución se revisa fuera del solver: se recalculan la cobertura, la capacidad, la flota y el costo. La regla debe ser *la misma* para todos los métodos (Sección 10).

4. La unidad es la instancia. Primero se promedian las réplicas de cada instancia y después se comparan los métodos instancia por instancia con la prueba de rangos con signo de Wilcoxon [17]. Se añaden intervalos bootstrap [6] y, cuando hay varias pruebas, la corrección de Holm [7].

Ejemplo resuelto: intuición de la prueba de Wilcoxon

En cinco instancias, la diferencia de gap A–B vale +0.30, +0.10, –0.05, +0.20 y 0. Se descarta el cero, se ordenan los valores absolutos y se les asignan rangos: 0.05 → 1, 0.10 → 2, 0.20 → 3 y 0.30 → 4. La suma de los rangos negativos es $W^- = 1$ y la de los positivos $W^+ = 9$. Si A y B fueran igual de buenos, ambas sumas serían parecidas; aquí B gana en casi todas. Con solo cinco instancias el p -valor es grande; con 60 instancias y $W^- = 0$, como en nuestra comparación A/B/E, es minúsculo.

5. Congelar antes de mirar. Antes de ejecutar se congela un *contrato*: un archivo con el código, los parámetros, las instancias, las semillas y las pruebas planeadas, identificado por su huella SHA-256. Cualquier cambio posterior exige una *enmienda* registrada. Además, el selector se validó en un *holdout*: 10 instancias X que no se miraron durante su diseño.

6. No afirmar equivalencia sin un margen. Una mediana de diferencias igual a cero o un p -valor grande no demuestran que dos métodos sean iguales. Sin un margen de equivalencia declarado de antemano, solo se reportan diferencias.

9. Resultados confirmatorios

9.1. A/B/E: casi óptimo, pero HGS es sistemáticamente mejor

Tabla 7: A/B/E con flota fija verificada (2s, 10 réplicas). Gap y tiempo son medias por instancia. G/E/P cuenta las instancias ganadas, empatadas y perdidas por SENDA. HGS se ejecutó en el mismo bloque y en la misma máquina (Sección 10).

Método	n	Gap medio	Mediana	Peor	Tiempo (s)	G/E/P
SEDA (portafolio)	60	0.111 %	0.000 %	1.172 %	1.58	0/37/23
HGS-CVRP	60	0.004 %	0.000 %	0.094 %	2.02	referencia

SEDA alcanza la BKS en la mayoría de las instancias y su gap medio es de una décima de punto. Aun así, no gana ninguna: empatada en 37 y HGS es mejor en 23 (Wilcoxon $W = 0$, $p = 2.4 \times 10^{-7}$). La Figura 6 muestra la diferencia en cada instancia y, en gris, las cifras publicadas en la versión anterior, cuyas tres “victorias” de SENDA eran soluciones con un vehículo de más.

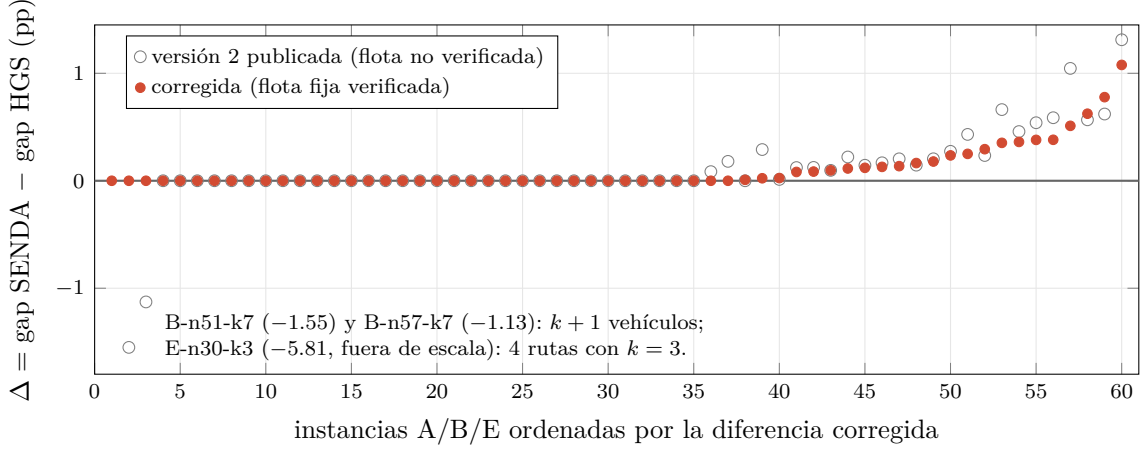


Figura 6: Diferencia de gap por instancia en A/B/E. Los valores negativos de la versión publicada (círculos grises) desaparecen al exigir la flota: con k vehículos, SENDA empata con HGS en esas tres instancias.

9.2. X: la arquitectura ligera deja de ser competitiva

Tabla 8: Instancias X (41, 4s, flota libre; campaña v2 del protocolo).

Método	Gap medio	Mediana	Peor	Tiempo (s)	G/E/P de SENDA
SENDA (variante X)	4.119 %	3.756 %	7.240 %	3.42	—
HGS-CVRP	1.450 %	1.452 %	3.101 %	4.17	0/0/41
FILO2	0.758 %	0.674 %	1.897 %	4.06	0/0/41

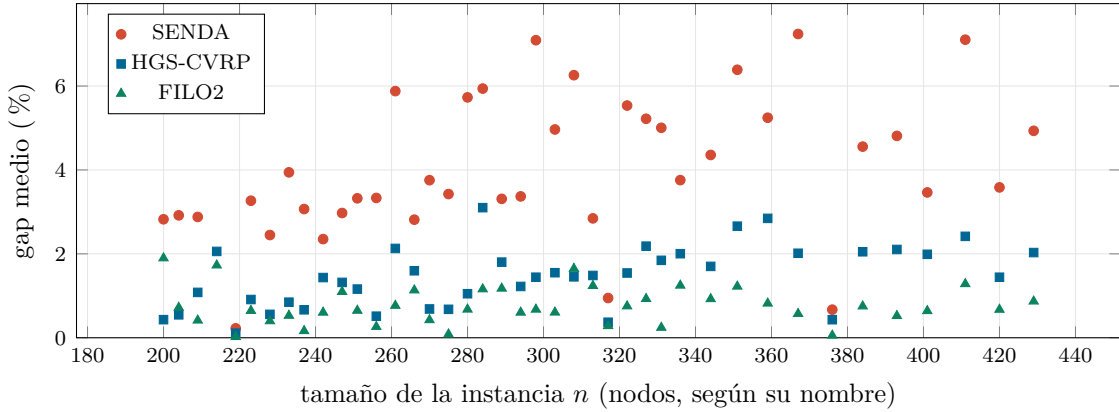


Figura 7: Gap por instancia en X. HGS y FILO2 son mejores en las 41 instancias ($p = 9.1 \times 10^{-13}$ en ambas comparaciones). La brecha no se debe a unos pocos casos atípicos.

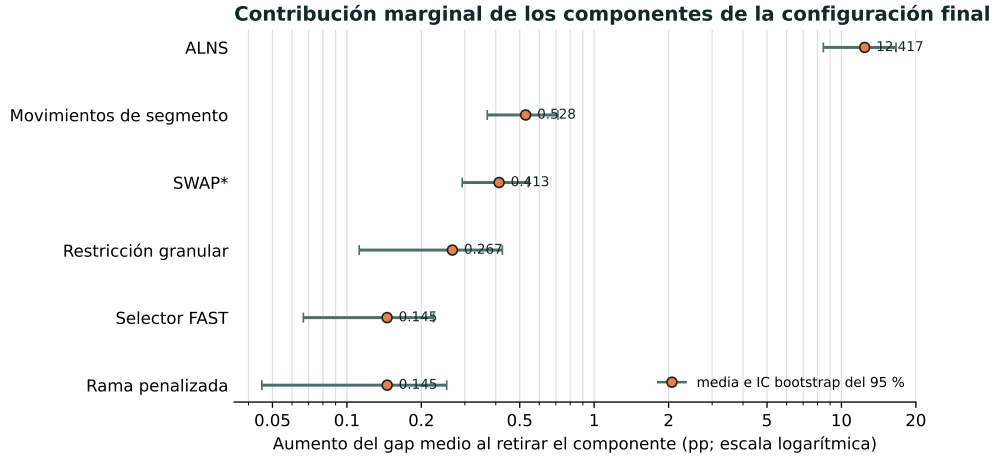
Tiempo frente a calidad. En una exploración anterior, no confirmatoria y con presupuesto por iteraciones —unas 18 veces más tiempo real que HGS—, SENDA llegaba a 1.08 % en las 31 instancias X de desarrollo, frente a 0.95 % de HGS. La brecha de X es, sobre todo, una cuestión de velocidad: la arquitectura necesita mucho más tiempo que HGS para alcanzar calidades similares. La Sección 7 muestra por qué: el costo de una iteración del ALNS crece al menos como n^2 .

9.3. Ablación: qué sostiene la calidad

Para saber qué componente importa, se retira uno a la vez con el mismo presupuesto (41 instancias X, 10 réplicas, corrección de Holm sobre seis comparaciones).

Tabla 9: Ablación por componente en X. Δ es el gap de la versión completa menos el gap sin el componente; un valor negativo favorece a la versión completa. p_H es el p -valor ajustado por Holm.

Componente retirado	G/E/P	Gap sin él	Δ (pp)	p_H
ALNS	40/1/0	16.702 %	-12.417	1.1×10^{-11}
Movimientos de segmento	36/0/5	4.813 %	-0.528	7.9×10^{-9}
SWAP*	38/0/3	4.698 %	-0.413	4.1×10^{-8}
Restricción granular	28/0/13	4.552 %	-0.267	5.7×10^{-3}
Selector FAST	29/0/12	4.430 %	-0.145	2.5×10^{-3}
Brazo penalizado	23/0/18	4.430 %	-0.145	4.8×10^{-2}



Valores positivos indican deterioro al retirar el componente; las seis comparaciones superan Holm.

Figura 8: Aumento del gap al retirar cada componente (escala logarítmica). Todos los intervalos bootstrap del 95 % quedan por encima de cero.

Idea clave: jerarquía de mecanismos

El ALNS es el motor: sin él, el gap se multiplica por cuatro. Los movimientos entre rutas (segmentos y SWAP*) forman un segundo nivel. El selector y el brazo penalizado son refinamientos de menor magnitud.

9.4. Holdout prospectivo

El selector y sus umbrales se congelaron, con su huella SHA-256, antes de ejecutar 10 instancias X que nunca se habían observado (X-n351-k40 a X-n429-k61). En ellas los gaps medios fueron 4.799 % para SENDA, 1.998 % para HGS y 0.740 % para FILO2. El orden entre métodos se conserva: el selector funciona como regla fuera de las instancias de diseño, pero no convierte a SENDA en el mejor solver de X.

10. Caso de estudio: una auditoría que corrigió nuestros propios resultados

10.1. Qué se publicó

La versión 2 reportaba en A/B/E un gap de 0.0120 % para SENDA frente a 0.0059 % para HGS, con 3 instancias ganadas por SENDA, 33 empates y 25 derrotas.

10.2. Cómo se detectó el problema

Al repetir la comparación en una campaña posterior aparecieron *gaps negativos*: soluciones de SENDA aparentemente mejores que el óptimo. Siguiendo la alarma de la Sección 2, se contaron las rutas de esas soluciones.

Tabla 10: Soluciones con costo inferior a la BKS en los datos del proyecto. Ninguna es un récord legítimo.

Instancia	BKS (k)	Solución	Rutas	Veredicto
B-n51-k7	1032 (7)	1016	8	usa un vehículo de más
B-n57-k7	1153 (7)	1140	8	usa un vehículo de más
E-n30-k3	534 (3)	503	4	usa un vehículo de más
X-n247-k50, X-n420-k130	—	—	—	inválidas, ya excluidas en v2

10.3. La causa

El programa que lanzaba SENDA en los experimentos (el *runner*) le indicaba “flota libre” incluso en A/B/E, y su verificación comprobaba la cobertura y la capacidad, pero no el número de rutas. HGS, en cambio, se ejecutó con la flota fija, y a FILO2 sí se le verificó la flota. En total, 327 corridas de SENDA con más de k rutas se aceptaron como válidas. Una revisión posterior de toda la carpeta del proyecto mostró que el mismo problema afecta a los perfiles de sensibilidad de A/B/E (CPU igualada y portafolio paralelo), cuyos gaps medios de SENDA también son negativos, y a la versión 1 del protocolo.

10.4. La corrección

1. Se repitió el análisis publicado a partir de los datos crudos y se obtuvo, byte a byte, la misma tabla de comparaciones. Así se confirmó que el reanálisis no introducía cambios propios.
2. Se aplicó la misma regla de flota a todos los métodos. En los datos originales, tres instancias se quedan sin ninguna réplica válida de SENDA, así que esos datos ya no sostienen la comparación.
3. Se repitió el experimento con el runner corregido (flota igual a k), el mismo presupuesto, las mismas semillas y HGS en el mismo bloque. Con la flota indicada, SENDA la respetó en las 630 corridas: el vehículo extra venía del runner, no del algoritmo. Ese es el resultado de la Tabla 7.

Lección metodológica: verificar igual a todos

Una comparación solo es justa si *todos* los métodos se verifican con la misma regla. Aquí, una diferencia de trato —flota fija para HGS, flota libre para SENDA— convirtió tres soluciones inválidas en “victorias”. Las señales de alarma existían: las familias B y E tenían gaps medios negativos. Faltaba mirarlas con sospecha. La corrección mantiene la conclusión cualitativa (SENDA es casi óptimo en A/B/E), pero cambia la magnitud de 0.012 % a 0.111 % y elimina toda victoria de SENDA.

Para reproducir

Datos originales: CSV canónico `f3a1164c85f4`. . . . Re-corrida con flota fija y verificada: contrato `fef9dd7ee521`. . . . Para repetir la verificación con sus propios resultados, compruebe en cada solución el número de rutas contra la flota de la instancia, además de la capacidad y la cobertura (Sección 13).

11. Discusión

Dos regímenes. En instancias de hasta cien clientes, SENDA queda a una décima de punto de la BKS en dos segundos; HGS es mejor, pero por poco. En instancias de 199 a 428 clientes, la brecha frente a HGS y FILO2 crece a entre 2 y 3 puntos. La cascada sugiere por qué: en X, cada brazo dispone de poco más de un segundo, y en ese tiempo su ALNS apenas mejora lo que ya logró la búsqueda local (4.971 % frente a 5.244 % en las soluciones factibles). El tiempo, no la construcción, es el cuello de botella.

Significancia estadística frente a relevancia práctica. Una diferencia de 0.1 % en distancia puede ser irrelevante o importante según la operación. Cuando el ruteo se integra con restricciones de carga tridimensional, ventanas de tiempo y reglas operativas, esas restricciones suelen pesar más que las diferencias de ruteo puro. Cuantificarlo en el problema integrado es trabajo futuro; este documento se limita al CVRP clásico.

Limitaciones. (i) La campaña principal y la re-corrida con flota fija se ejecutaron en máquinas distintas. Dentro de cada comparación todos los métodos comparten máquina, pero no deben mezclarse cifras de fuentes distintas. (ii) El holdout contiene solo 10 instancias de una familia. (iii) En X, la cascada usa el portafolio de A/B/E y no la variante X de la campaña principal (4.150 % frente a 4.119 %). (iv) La cascada se midió con el binario de la ablación, no con el confirmatorio.

12. Conclusiones

SENDA muestra que una arquitectura de trayectoria única por corrida, auditable y sin población alcanza una calidad casi óptima en instancias clásicas pequeñas: 0.111 % de gap en 2s con la flota verificada, aunque HGS es igual o mejor en todas ellas. En instancias X, HGS y FILO2 mantienen una ventaja clara. La cascada y la ablación coinciden en que el ALNS es el mecanismo dominante, que los movimientos entre rutas forman un segundo nivel y que el selector y el brazo penalizado afinan el resultado.

Metodológicamente, el proyecto deja una enseñanza que vale más que cualquier cifra: la verificación debe ser uniforme, los gaps negativos son alarmas y los resultados propios también deben auditarse. Corregirlos públicamente no debilita el trabajo: lo hace creíble.

Trabajo futuro. (1) Dar más tiempo efectivo al ALNS en instancias grandes, por ejemplo evitando que cada brazo repita la construcción y el pulido. (2) Medir el impacto de estas diferencias en el problema integrado con carga tridimensional. (3) Ampliar el holdout a otras familias y hasta $n = 1000$.

Para reproducir: artefactos

Campaña v2 (X, ablación, holdout)	contrato 36f29f1f1795..., CSV f3a1164c85f4...
Corrección de flota	contrato fef9dd7ee521...
Cascada de calidad	contrato dd6b8817...; binario de ablación 4002751d...
Núcleo evaluado y HGS-CVRP	binario del núcleo d7921fe1...; binario de HGS-CVRP 4e53fb18...
Figuras de este documento	datos en paper/datos/ de este repositorio
Artefactos completos	archivo del autor; se comparten a solicitud (contacto@lattimex.com)

13. Pruébalo en su computadora

El motor de ruteo de LATTIMEX usa una versión posterior de SENDA (motor 1.6.0): conserva sus componentes y agrega el portafolio en paralelo de la Sección 5.8. Las cifras de esta guía se midieron con el binario d7921fe1..., no con el del repositorio. El motor está publicado con licencia MIT en <https://github.com/Erick-MF28/lattimex>. En el repositorio el núcleo es `native/send_core.cpp`; alrededor de él hay una tubería que lo convierte en un planificador real: capacidad efectiva, acomodo tridimensional de las cajas, mediación con flota fija y plan de descarga por puertas. Los programas de comparación (HGS-CVRP, FILO2 y los scripts de las campañas) no forman parte del repositorio.

1. Instalar y comprobar el motor. Se necesita Python 3.10 o superior y un compilador de C++ (en Windows basta `pip install ziglang`).

```
git clone https://github.com/Erick-MF28/lattimex.git
cd lattimex
pip install -r requirements.txt
python -m lattimex build          # compila el núcleo y el empacador 3D
python -m lattimex probe         # huella determinista (ver abajo)
python -m unittest discover -s tests/engine
```

La huella determinista resume una corrida fija de 3000 iteraciones. Depende de la biblioteca estándar de C++, porque `std::shuffle` y las distribuciones aleatorias se implementan distinto en cada una: 6457b2dd44660773 con `zig/libc++` (el instalador de Windows) y 285bf1758e5dd6a1 con `g++/libstdc++` (Linux y MinGW). Si su compilación da la huella de su biblioteca, su binario calcula exactamente lo mismo que el publicado, aunque el archivo no sea idéntico byte por byte.

2. Verlo trabajar sobre calles reales. El motor construye su propio mapa a partir de OpenStreetMap y lo sirve a la interfaz gráfica, el LATTIMEX Planner, que se abre en el navegador:

```
python -m lattimex map --bbox 20.47,-100.52,20.82,-100.22 --name queretaro
python -m lattimex serve          # luego abra https://lattimex.com/planner
```

En la sección Guías del Planner, el botón *Cargar ejemplo* agrega 300 entregas sintéticas en Querétaro, seis vagonetas y cinco operarios; *Optimizar* calcula las rutas y el acomodo de cada caja. Los cálculos y los datos se quedan en su computadora.

3. Repetir una comparación de forma justa. Para medir el núcleo en instancias de referencia (CVRPLIB) contra otro solver, obtenga ese solver de su fuente oficial y respete las reglas de la Sección 8:

1. el mismo reloj de pared para todos los métodos, en la misma máquina y con un solo hilo;
2. al menos 10 réplicas con semillas distintas por instancia;
3. la misma flota para todos, fijada de antemano;
4. un verificador independiente que recalcule distancia, capacidad, cobertura y número de rutas;
5. comparaciones pareadas por instancia y corrección por comparaciones múltiples (Holm).

Un gap negativo frente a la mejor solución conocida es, casi siempre, una alarma: revise primero la flota y el redondeo de distancias.

Idea clave

El código abierto permite que cualquiera repita las mediciones. Si sus resultados difieren de los de esta guía, escríbanos: una discrepancia bien documentada es una contribución.

Referencias

- [1] Luca Accorsi and Daniele Vigo. Routing one million customers in a handful of minutes. arXiv:2306.14205, 2023.
- [2] Philippe Augerat, José Manuel Belenguer, Enrique Benavent, Ángel Corberán, Denis Naddef, and Giovanni Rinaldi. Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical Report RR 949-M, Institut National Polytechnique de Grenoble, 1995.
- [3] Nicos Christofides and Samuel Eilon. An algorithm for the vehicle-dispatching problem. *Journal of the Operational Research Society*, 20(3):309–318, 1969.
- [4] CVRPLIB. CVRPLIB: Capacitated vehicle routing problem library. <http://vrp.atd-lab.inf.puc-rio.br/>, 2026. Accedido en junio de 2026.
- [5] George B. Dantzig and John H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.
- [6] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [8] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [9] Jan Karel Lenstra and Alexander H. G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2):221–227, 1981.
- [10] Eduardo Queiroga, Ruslan Sadykov, Eduardo Uchoa, and Thibaut Vidal. 10,000 optimal CVRP solutions for testing machine learning based heuristics. *AAAI Workshop on Machine Learning for Operations Research*, 2021.
- [11] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.
- [12] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP98*, volume 1520 of *Lecture Notes in Computer Science*, pages 417–431. Springer, 1998.
- [13] Paolo Toth and Daniele Vigo. The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15(4):333–346, 2003.
- [14] Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.
- [15] Thibaut Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140:105643, 2022.

- [16] Thibaut Vidal, Teodor Gabriel Crainic, Michel Gendreau, Nadia Lahrichi, and Walter Rei. A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. *Operations Research*, 60(3):611–624, 2012.
- [17] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

A. Glosario

BKS Mejor solución conocida de una instancia de referencia.

Gap Diferencia relativa entre el costo obtenido y la BKS (ecuación 2).

Óptimo local Solución que ningún movimiento de un vecindario dado puede mejorar.

Réplica Ejecución independiente con otra semilla. Aquí se usan 10 por instancia.

Brazo Una búsqueda independiente dentro de un ensayo; el portafolio tiene dos o tres.

Tiempo de pared Tiempo real transcurrido, medido con un reloj; se distingue del tiempo de CPU.

Construcción Procedimiento que genera la primera solución completa.

Búsqueda local Aplicación repetida de cambios pequeños que mejoran la solución.

Restricción granular Limitar los movimientos a pares de clientes cercanos.

ALNS Búsqueda adaptativa de vecindarios grandes: destruye y repara partes de la solución con operadores cuyos pesos aprenden.

Recocido simulado Regla que acepta soluciones peores con una probabilidad que disminuye con el tiempo.

Regret-2 Reparación que inserta primero al cliente que más perdería si se le deja para después.

SWAP* Intercambio de dos clientes entre rutas, cada uno en su mejor posición de la otra ruta.

Portafolio Configuración de SENDA evaluada en A/B/E: rama base, rama FAST y brazo penalizado.

Brazo penalizado Búsqueda que tolera excesos de capacidad temporales a cambio de una penalización.

Contrato Archivo congelado con código, parámetros, instancias y pruebas, identificado por su huella SHA-256.

Holdout Conjunto de instancias reservado y no observado durante el diseño.

Holm Corrección para comparaciones múltiples que controla el error de la familia de pruebas.

O grande Notación que describe cómo crece el número de operaciones de un algoritmo con el tamaño de la entrada (Sección efsec:complejidad).

B. Regla literal del selector FAST

Algoritmo 2 Configuración adicional elegida por FAST a partir de $\phi(I)$

Entrada: n , k , presión de carga ℓ , CV de demanda c_q , CV radial c_r , concentración angular α , densidad η

```
1: si  $n < 150$  entonces
2:   si  $n \geq 70$  entonces devolver granular_k25
3:   si  $60 \leq n < 70$  entonces
4:     si  $\eta \geq 0.0085$  o  $\alpha < 0.35$  entonces devolver large_destroy_k25
5:     si  $\alpha \geq 0.80$  entonces devolver long_10k_k25
6:     si  $k \geq 10$  entonces devolver granular_k40
7:     si  $c_q < 0.50$  entonces devolver long_10k_k25
8:     devolver baseline
9:   fin si
10:  si  $45 \leq n \leq 65$  y  $\ell < 0.93$  y  $c_q < 0.50$  entonces devolver medium_destroy_k25
11:  devolver baseline
12: fin si
13: si  $c_q \geq 0.50$  y ( $n \leq 245$  o  $k \geq 70$ ) entonces devolver large_destroy_k25
14: si  $k \leq 16$  y  $[(c_q < 0.30$  y  $c_r < 0.38)$  o  $(c_q < 0.70$  y  $\alpha \geq 0.88)]$  entonces devolver granular_k25
15: si  $k \geq 55$  y  $c_q \geq 0.45$  y  $\alpha \geq 0.90$  entonces devolver granular_k25
16: si  $n \geq 300$  y  $c_q \geq 1.0$  entonces devolver baseline
17: devolver long_10k_k25
```

Si la configuración elegida es **baseline**, el ensayo ejecuta solo la rama base y su brazo penalizado. En otro caso ejecuta la rama base, la rama elegida y el brazo penalizado de esta última.

C. Errata respecto a la versión 2

- **A/B/E:** “SENDA 0.0120 % frente a HGS 0.0059 %, 3/33/25, $p = 0.00406$ ” se sustituye por “SENDA 0.111 % frente a HGS 0.004 %, 0/37/23, $p = 2.4 \times 10^{-7}$ (60 pares, flota fija verificada)”.
- **Verificación:** la afirmación de que se verificó la flota de cada solución no era cierta para SENDA en la campaña v2; se corrigió en la re-corrida.
- **Gates:** con la flota verificada, el gate de la comparación A/B/E de la campaña v2 falla; los de X, ablación y holdout pasan sin cambios.
- **Figura del mapa por instancia:** las tres celdas favorables a SENDA correspondían a soluciones con $k + 1$ vehículos (Figura 6).